

DSA STUDY BLUEPRINT SERIES

Kadane's Algorithm Blueprint

Maximum Contiguous Subarray Sum Optimization

Section 03 • C++ Core Data Structures & Algorithms

The Maximum Subarray Sum Problem

Finding the largest contiguous sum in a one-dimensional array

Brute Force Approach $O(N^2)$

Compute the sum of all possible subarrays and keep track of the maximum.

- Inner loop counts sums from index i to j .
- Time Complexity: $O(N^2)$.
- Space Complexity: $O(1)$.
- Extremely slow for large inputs (e.g. $N = 10^5$).

Kadane's Optimization $O(N)$

Linear traversal tracking local maximum sums and global maximum sums.

- Single loop through the array.
- Time Complexity: $O(N)$.
- Space Complexity: $O(1)$.
- Resets subarray tracking when current sum drops below zero.

Algorithmic Mechanics & Variables

Tracking local and global progress using two state variables

State Variables

- **curr_sum**: Sum of the subarray currently being built.
- **max_so_far**: Global maximum subarray sum seen so far (initialized to `INT_MIN`).

The Reset Law:

If `curr_sum` drops below 0, reset it to 0. A negative sum will always decrease the sum of any subsequent subarray.

Visual Tracing Walkthrough

Array: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

Element	curr_sum	max_so_far	Action
-2	-2 → 0	-2	Reset curr_sum
1	1	1	Update max_so_far
-3	-2 → 0	1	Reset curr_sum
4	4	4	Update max_so_far
-1	3	4	Keep building
2	5	5	Update max_so_far
1	6	6	Update max_so_far (Ans)

Kadane's Algorithm Code

Standard implementation in C++

```
#include
#include
using namespace std;

int maxSubarraySum(int arr[], int size) {
    int max_so_far = INT_MIN; // Global maximum
    int curr_sum = 0;         // Local running sum

    for (int i = 0; i < size; i++) {
        curr_sum += arr[i];

        if (curr_sum > max_so_far) {
            max_so_far = curr_sum;
        }

        if (curr_sum < 0) {
            curr_sum = 0; // Reset negative sum contribution
        }
    }

    return max_so_far;
}
```

Edge Cases & Extended Challenges

Advanced scenarios and modifications of Kadane's

Handling All-Negative Arrays

- If the array contains only negative numbers, `max_so_far` will capture the largest single negative element (closest to 0).
- Example: `[-3, -2, -5]` returns `-2` because `max_so_far` updates before `curr_sum` resets.

Extended Challenges

- **Print Subarray Range:** Maintain start/end index indices. Update them whenever `max_so_far` is updated.
- **Circular Subarray Sum:** Maximum sum can wrap around. Solve as `max(Normal Max, Total - Min Subarray Sum)`.